

Streaming Graph Neural Networks via Continual Learning

Junshan Wang

Key Laboratory of Machine Perception, Ministry of
Education, Peking University
Beijing, China
wangjunshan@pku.edu.cn

Yi Wu

Peking University
Beijing, China
reyn19990619@pku.edu.cn

Guojie Song*

Key Laboratory of Machine Perception, Ministry of
Education, Peking University
Beijing, China
gjsong@pku.edu.cn

Liang Wang

Alibaba group
Beijing, China
liangbo.wl@alibaba-inc.com

ABSTRACT

Graph neural networks (GNNs) have achieved strong performance in various applications. In the real world, network data is usually formed in a streaming fashion. The distributions of patterns that refer to neighborhood information of nodes may shift over time. The GNN model needs to learn the new patterns that cannot yet be captured. But learning incrementally leads to the catastrophic forgetting problem that historical knowledge is overwritten by newly learned knowledge. Therefore, it is important to train GNN model to learn new patterns and maintain existing patterns simultaneously, which few works focus on. In this paper, we propose a streaming GNN model based on continual learning so that the model is trained incrementally and up-to-date node representations can be obtained at each time step. Firstly, we design an approximation algorithm to detect new coming patterns efficiently based on information propagation. Secondly, we combine two perspectives of data replaying and model regularization for existing pattern consolidation. Specially, a hierarchy-importance sampling strategy for nodes is designed and a weighted regularization term for GNN parameters is derived, achieving greater stability and generalization of knowledge consolidation. Our model is evaluated on real and synthetic data sets and compared with multiple baselines. The results of node classification prove that our model can efficiently update model parameters and achieve comparable performance to model retraining. In addition, we also conduct a case study on the synthetic data, and carry out some specific analysis for each part of our model, illustrating its ability to learn new knowledge and maintain existing knowledge from different perspectives.

CCS CONCEPTS

• **Networks** → **Online social networks**; *Network structure*; • **Theory of computation** → *Online learning algorithms*.

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions.acm.org.

CIKM '20, October 19–23, 2020, Virtual Event, Ireland

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-6859-9/20/10...\$15.00

<https://doi.org/10.1145/3340531.3411963>

KEYWORDS

graph neural networks; continual learning; streaming networks

ACM Reference Format:

Junshan Wang, Guojie Song, Yi Wu, and Liang Wang. 2020. Streaming Graph Neural Networks via Continual Learning. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM '20)*, October 19–23, 2020, Virtual Event, Ireland. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3340531.3411963>

1 INTRODUCTION

Network structures are widely seen in the real world nowadays. As a method that maps nodes into vectors and preserves local structures, network embedding plays an important role in many applications such as node classification and recommendation, which can be categorized in two types: SkipGram models [19] and GNNs [8, 9]. Among them, GNNs take advantages of deep learning techniques and are capable of combining node features and local structures together, achieving great improvements than traditional models. But most of the existing methods are based on the assumption that networks are static. The model is trained on the entire network, and parameters will not be updated after training.

However, network data is usually formed in a streaming fashion and most real-world networks are continuously evolving over time, which are referred as streaming networks. For example, edges are added or removed in social networks, and attributes of nodes may also change over time. The dynamics give rise to some *new patterns* [28], referring to the neighborhood information not captured by the GNN, and at the same time, many *existing patterns* that have been learned by the GNN in the previous network still maintain. For example, new research fields may grow up in citation networks and in different years, users may generate distinct types of social circles. In order to get the updated representations, it is necessary to learn both new and existing patterns, but retraining the GNN once the network changes results in high computational complexity.

There are some existing researches on streaming network embedding specially, focusing on either capturing network evolutionary patterns or efficiently updating node representations. Our model belongs to the latter type, with typical examples including DANE [11] and DNE [5]. They design strategies to update node representations in a transductive setting, which lack scalability and cannot be migrated to the GNN architecture. There are some naive incremental learning approaches for GNNs. On one hand, GNNs can efficiently

generate new representations by aggregating features from neighborhood based on an inductive framework but the performance will gradually degrade as parameters do not absorb knowledge from current networks. On the other hand, applying traditional online learning approaches causes *catastrophic forgetting* problem. That is, existing patterns will be overwritten and forgotten abruptly when new patterns are incorporated, resulting in the inability to obtain good representations for previous nodes through the current GNN. Therefore, training GNNs incrementally to ensure that models are updated to learn new patterns and consolidate existing patterns simultaneously during network evolution is meaningful.

In supervised learning tasks, data is sampled identically and independently from a fixed distribution, and its learning goal is to minimize the empirical loss for all data. *Continual Learning*, also called as life-long learning, is a learning method in a streaming scenario. In the real world, data in different distributions and from different domains arrives in a streaming manner. Storing all data and retraining on each time step is not feasible on large-scale data. Thus, the learning goal of continual learning is to minimize the loss of data under the current task without interfering the data that has been learned before. When the data distribution shifts over time, it can avoid the catastrophic forgetting of previous tasks. Hence, we adopt continual learning to address the problem for training GNNs incrementally because network distributions may shift in streaming networks. And our goal is to develop an efficient algorithm that can not only capture new patterns from current networks but also consolidate existing patterns from previous networks. Yet, the following challenges still remain:

- New patterns need to be detected. Due to the complexity of network structures, it is difficult to estimate whether a node is seriously affected and corresponds to a new pattern. Besides, any node of the entire network may be affected, but calculating the influenced degrees of all nodes is time-consuming and not feasible on the large-scale networks.
- Existing patterns need to be consolidated. It is unrealistic to save and retrain the entire network, and an alternative is to sample and replay only a small number of nodes. But on one hand, naive sampling strategies are limited by the instability and poor effectiveness. On the other hand, it may lead to the overfitting problem and lacks additional constraints to improve generalization of knowledge maintaining.

Given these challenges, we propose a streaming GNN model via continual learning in streaming networks. When distributions of networks shift over time, our model can capture new patterns incrementally and consolidate existing information. Firstly, in order to learn new patterns, we propose an approximate method based on the propagation process of changes in the network to mine affected nodes efficiently. Then the existing knowledge of the network is modeled and maintained from two complementary perspectives. To improve the effectiveness and stability of data replaying, we design a hierarchy-importance sampling strategy to store some nodes in the memory. At the same time, in order to solve the overfitting problem caused by the small number of replayed nodes, a weighted regularization method for model parameters based on Fisher information is derived. We evaluate our model on the real-world and

synthetic graphs. Our model outperforms other incremental baselines and achieves lower time complexity than retraining models. In addition, we show that our model can alleviate the problem of catastrophic forgetting through a case study on synthetic data. Each part of the model is also analyzed in detail, further illustrating the superiority of our model in detecting and learning new patterns and consolidating existing patterns.

Overall, our contributions are as follows:

- We propose a streaming GNN framework via continual learning that can update model parameters efficiently and generate up-to-date node representations when the network evolves and its distribution shifts over time.
- We design an approximation algorithm to detect new patterns. A new data sampling method and a model regularization method are proposed and combined to consolidate the existing patterns in the streaming network.
- We conduct experiments on four data sets which show that our streaming GNNs can be updated incrementally with less accuracy lost than other models.

2 RELATED WORK

2.1 Network Embedding

Traditional network embedding methods can be categorized into two types. SkipGram models such as DeepWalk [19] and LINE [23] are limited to their transductive setting. GNN models such as GCN [9], GraphSAGE [8], GAT [25] and GIN [30] learn node representations by aggregating neighborhood features at each layer and often achieve outstanding performance in tasks. But all the aforementioned methods are only designed for static networks.

As real-world graph data often appears streamingly, existing works in dynamic networks mainly focus on two aspects. Some of them capture temporal information along with local structure so as to improve the expressive ability of the model, with typical examples like DynamicTriad [34], HTNE [36], EPNE [27], EvolveGCN [18], DynamicGCN [4], DGNN [14] and ST-GCN [31]. But these models have the shortcoming of high computation complexity as they need to train a new model at each time step. Other methods study how to efficiently update node representations when networks change. DANE [11] proposes online learning for dynamic attributed networks with a spectral embedding manner. DNE [5] and [12] consider that it is unnecessary to update representations of all nodes but only the most affected nodes need to be adjusted during the evolution of networks. But they are all transductive models and cannot transfer to GNNs. As traditional GNNs are inductive models, new representations can be obtained directly but the scalability of models is limited as parameters are fixed after training.

2.2 Continual Learning

The key point of continual learning is to consolidate the knowledge learned from previous data in a streaming scenario to avoid catastrophic forgetting. Recent methods can be divided into three categories. A first family are the regularization-based methods by a parameter regularization, including EWC [10], SI [33], MAS [1] and VCL [17]. Their implementation is simple and enjoy the beauty of the Bayesian framework. A second family are the replay-based methods that old examples reproduced from a replay buffer such

as GEM [13] and iCaRL [20], or a generative model [22] will be used for retraining. The idea of replaying data was proposed very early, but it is still widely used and performs very well in practical applications. The last family are parameter isolation that isolates parameters from different tasks explicitly [15] and draws more on the idea of transfer learning. All these methods follow the task-based setting, that is, the clear task boundaries are utilized to help store or process data or models from different previous tasks.

However, task boundaries are often not available in real-world tasks in streaming scenarios so that the above methods are no longer applicable. [2] and [3] both propose methods that extend continual learning to online setting on streaming data, which are based on regularization and replay respectively. But they only work on independent data like images but cannot deal with streaming data on complex structures like networks.

3 PRELIMINARIES

In this section, we introduce the basic idea of GNNs and the problem definition of streaming GNNs, and then give a few simple but insufficient algorithms, which will be used as comparison methods in our experiments.

Let $G = (A, X)$ be an attributed graph where $A_{n \times n} \in \{0, 1\}$ is the adjacency matrix and $X_{n \times d} \in [0, 1]$ is the node features. Let $V = \{v_i\}_{i=1}^n$ represent the set of nodes and $E = \{e_{ij}\}_{i,j \in V}$ represents the set of edges.

Graph Neural Networks (GNNs) are a general architecture for neural networks on graphs to generate node representations by an aggregator for neighborhood information. Here, we follow the established work GraphSAGE [8] where the layer l -th is defined as:

$$h_v^l = \sigma(W^l \cdot \text{MEAN}(\{h_u^{l-1}\} \cup \{h_v^{l-1}\}, \forall u \in \mathcal{N}(v))), \quad (1)$$

where $\mathcal{N}(v)$ specifies neighborhood of node v and σ is the activation function. We then use node representations in cross-entropy loss for node classification as

$$L(\theta; A, X) = \frac{1}{n} \sum_{v \in V_{\text{label}}} l(\theta; v) = \frac{1}{n} \sum_{v \in V_{\text{label}}} l(\text{softmax}(h_v^L), y_v), \quad (2)$$

where $V_{\text{label}} \subset V$ is a subset of labeled nodes, y_v is the label of node v and L is the number of layers of the GNN.

A **Streaming Network** is represented as $G = (G^1, G^2, \dots, G^T)$, where $G^t = G^{t-1} + \Delta G^t$. $G^t = (A^t, X^t)$ is an attributed graph at time t and $\Delta G^t = (\Delta A^t, \Delta X^t)$ is the changes of node attributes and network structures at t . Notice that the changes contain newly added nodes, which are difficult to handle for many existing dynamic network representation learning methods.

Streaming Graph Neural Networks (Streaming GNNs) are extensions of traditional GNNs in a streaming setting. Given a streaming network G , the goal is to learn $(\theta^1, \theta^2, \dots, \theta^T)$ where θ^t is the parameters of the GNN at t that can generate satisfactory h_v^L representation for any node $v \in G^t$.

In the streaming graph data, the network is continuously evolving, resulting in changes in the patterns, referring to the neighborhood information of the node captured by GNNs, including network structures and node attributes. The changes of these patterns result in that node representations obtained through GNNs need to be updated. We discuss three simple schemes as follows.

- **Pretrained GNN:** As GNNs are inductive models, a straightforward idea to generate representations for the previously unseen nodes and changed nodes on G^t is to use a pre-trained GNN model. But if the patterns of these nodes are different from knowledge in the pre-trained model, then good representations cannot be obtained.
- **Retrained GNN:** Another naive solution is to train a new GNN on the whole graph data over again. It may has high performance but suffers from affordable time and space cost if the network becomes large as time goes by.
- **Online GNN:** A better solution is to train on ΔG^t using θ^{t-1} to initialize, which is called online learning. But if the patterns in ΔG^t are different from θ^{t-1} , catastrophic forgetting will occur. Knowledge of θ^{t-1} may be abruptly lost and degraded representations of nodes in G^{t-1} will be obtained.

It can be seen that the above schemes all have great limitations when dealing with large-scale streaming graph data whose pattern distribution shifts over time. So in the next section, we propose our streaming GNN model to avoid all the deficiencies of the above schemes.

4 STREAMING GNNs VIA CONTINUAL LEARNING

In this section, we first introduce the basic framework of our model, called **ContinualGNN**, which can learn new patterns that appear in the network and consolidate existing patterns simultaneously. Next, we propose an efficient approximation algorithm to detect the emergence of new patterns in streaming networks, and a multi-view based method to consolidate existing patterns.

4.1 Model Framework

Due to the limitations of all naive methods as discussed in Section 3, we propose streaming GNNs via continual learning that has been proved to be effective and memory efficient to mitigate catastrophic forgetting. That is to say, through continual learning, our model can maintain the knowledge of previous tasks and current tasks at the same time in a streaming scenario. In the followings, we give some concepts corresponding to continual learning in the context of streaming networks, and then on this basis, we present the general goal of our model.

- **Current Tasks D^t :** A current task is defined as training a GNN with parameters θ^t on the new network ΔG^t . But as nodes and edges are not independent in networks, ΔG^t may influence the neighborhood of other nodes. It is more proper to define the current task as affected parts of the network.
- **Previous Tasks $(D^1, \dots, D^{t-1})$:** Previous tasks are defined as training GNNs on the previous changes of the network $(\Delta G^1, \Delta G^2, \dots, \Delta G^{t-1})$. If there are lots of time steps in streaming data, it will be unaffordable to preserve information for each previous task at each snapshot. So we replace streaming data with an integrated network G^{t-1} so that only a single task will be considered for knowledge consolidation.

Furthermore, following [10], we explain our goal from a probabilistic perspective. Since we want to learn a GNN parameterized by θ on $G^t = G^{t-1} + \Delta G^t$ with a supervised loss, we use ΔG^t and

G^t to represent the nodes and the corresponding labels to simplify formulas and then the conditional probability can be computed by using Bayes' rule:

$$\log p(\theta|G^{t-1}, \Delta G^t) = \log p(\Delta G^t|\theta) + \log p(\theta|G^{t-1}) - \log p(\Delta G^t), \quad (3)$$

where the first term $\log p(\Delta G^t|\theta)$ is the log likelihood for the task on ΔG^t . The second term $\log p(\theta|G^{t-1})$ is the posterior distribution that absorbs information learned from the task on G^{t-1} . The last term $\log p(\Delta G^t)$ is a constant. Then we have the general loss function of our ContinualGNN in streaming network at time t as

$$L = L_{new} + L_{existing} \quad (4)$$

where the first term L_{new} is the loss function on the influenced parts of networks. The second term $L_{existing}$ aims to consolidate patterns on previous network data. The following sections discuss how to design these two loss functions separately.

4.2 Detection for New Patterns

In a streaming scenario, network changes lead to the emergence of new patterns that was not captured by the previous GNN. The GNN need to be updated, which corresponds to the current task in continual learning. But it is insufficient to consider that the current task is to train the nodes in ΔG^t . On the one hand, these nodes may not contain new patterns if the changes are small. For example, when the neighborhood of a node is very stable, adding an edge will not have a great impact on its representation and retraining these nodes is a waste of time. On the other hand, new patterns may still appear in existing nodes. Considering that the network data is not independent of each other, some changes may affect other parts of the network. Any node may have new pattern during the network change, and these nodes also need to be retrained. So we should study how to mine the nodes that may have new patterns.

First of all, in order to solve the above problem, we give the definition of new patterns related to the influenced degree of nodes. If nodes are greatly affected, they may contain new patterns. The influenced degree of a node is defined by a *scoring function* which is the delta of representations obtained from the GNN between two adjacent snapshots. Then the set of influenced nodes, corresponding to the new patterns, is generated according to the scores as

$$I(\Delta G^t) = \{u | \|\Delta \mathbf{h}_u^{t,L}\| = \|\mathbf{h}_u^{t+\Delta t,L} - \mathbf{h}_u^{t,L}\| > \delta\}, \quad (5)$$

where δ is a threshold value and a hyper-parameter. A smaller δ allows more nodes to be treated as new patterns, and vice versa. However, it is time-consuming to calculate the scoring function for all nodes at each time step. In fact, this is not necessary because only the neighbors within the order L are affected. An improved idea is to use breadth-first-search (BFS) to find all the L -order neighbors and calculate their scoring function, but the complexity is still related to the size of the neighborhood of ΔG^t .

Secondly, we design a fast approximation algorithm for the scoring function based on the information propagation of the changes, and the time complexity is only related to ΔG^t . In the following, we first present a lemma, and then based on the lemma, give and analyze our approximate algorithm.

LEMMA 4.1. When attributes of node v_i are changed from $\mathbf{x}_i$ into $\mathbf{x}_i + \Delta \mathbf{x}_i$, the score of node v_u can be calculated as:

$$\Delta \mathbf{h}_u^{t,L} = f_u^{t,L}(\Delta \mathbf{x}_i \tilde{\mathbf{W}}), \quad (6)$$

where $\tilde{\mathbf{W}} = \prod_{l=1}^L \mathbf{W}^l$ and $f_u^{t,L}$ is a function calculated as

$$f_u^{t,L} = \frac{1}{d_u^t} \left(\sum_{u' \in N^t(u)} f_{u'}^{t,L-1} \right), l > 0 \quad (7)$$

$$f_i^{t,0} = 1, f_u^{t,0} = 0, u \neq i$$

where d_u^t and $N^t(u)$ are the degree and neighbors of v_u at time t .

PROOF. Following [35], we can use a surrogate model that still captures the idea of convolutions to accelerate the computation of scoring function. The non-linear activation function is replaced with a linear function so that parameters $\mathbf{W}^l$ can be absorbed into a matrix $\tilde{\mathbf{W}}$ that equals to the production of $\mathbf{W}^l$ at each layer. Then, $\Delta \mathbf{h}_u^{t,L}$ can be calculated as

$$\begin{aligned} \Delta \mathbf{h}_u^{t,L} &= \mathbf{h}_u^{t+\Delta t,L} - \mathbf{h}_u^{t,L} \\ &= \frac{1}{d_u^t} \left(\sum_{u' \in N^t(u)} \Delta \mathbf{h}_{u'}^{t,L-1} \right) \mathbf{W}^L \\ &= \dots = f_u^{t,L} \Delta \mathbf{x}_i \tilde{\mathbf{W}}, \end{aligned} \quad (8)$$

where $f_u^{t,L}$ reflects the influence of node v_i on v_u by propagating information through the network and is only related to node degrees. Node v_i will have a greater influence on v_u if there are more paths from v_i to v_u , and the degrees of other nodes on these paths are smaller. $\square$

According to Lemma 4.1, we discuss how to estimate the influenced degree of nodes efficiently from two perspectives.

- We first discuss the method of calculating $\Delta \mathbf{h}_u^{t,L}$ under the simplest network change, that is, when the attributes of a node change. Compute $\Delta \mathbf{x}_i \tilde{\mathbf{W}}$ once. Then implement BFS from node v_i and compute $f_u^{t,L}$ iteratively, which is only related to degrees of nodes at time t . The search will iterate over L rounds. Finally, after $\Delta \mathbf{x}_i \tilde{\mathbf{W}}$ and $f_u^{t,L}$ are obtained, $\Delta \mathbf{h}_u^{t,L}$ can be calculated according to Equation 6.
- We then consider the more general and complex situation in the network, that is, when the structures related to more than one node change. In this case, we adopt an approximate idea that the structural changes are only considered once during the information propagation process of GNNs. From the implementation point of view, $\Delta \mathbf{h}_i^1$ is computed and regarded as $\Delta \mathbf{x}_i$, which is converted to the situation where the node attribute changes. Then use a similar method as before to start a BFS from the changed node set, iteratively calculate $f_{u,i}^{t,L}$ of each possible affected node v_u affected by node v_i , and iterate L rounds. In the end, the influenced degree of each node v_u is quickly approximated, which is the summation of the influenced degree of each v_i :

$$\Delta \mathbf{h}_u^{t,L} \approx \sum_{v_i \in \Delta V^t} f_{u,i}^{t,L} \|\Delta \mathbf{h}_i^{t,1}\|, \quad (9)$$

where ΔV^t is denoted as the node set of ΔG^t .

The complexity of the original methods is $O(|V| \times \tilde{m})$ where $\tilde{m}$ is the complexity to compute $\tilde{\mathbf{W}}$, while the complexity of our approximation algorithm is reduced to $O(|\Delta V| \times \tilde{m} + |N_{\Delta V}^L| \times |\Delta V|)$, where $|N_{\Delta V}^L|$ is the number of L -hop neighbors of the node set ΔV .

Finally, according to the above definition and approximate method, we obtain the set of influenced node $\mathcal{I}(\Delta G^t)$ efficiently and then have the objective that replays these nodes to learn new patterns for the GNN at time t :

$$L_{new} = \sum_{v_i \in \mathcal{I}(\Delta G^t)} l(\theta; v_i). \quad (10)$$

4.3 Preservation for Existing Patterns

In a streaming network, when the GNN learns new knowledge, patterns in the historical tasks needs to be consolidated so as to avoid catastrophic forgetting. In the field of continual learning, one way to consolidate historical knowledge is to replay samples. But replaying all nodes at each snapshot is time-consuming. An alternative that a small number of nodes are saved and replayed is prone to overfitting problems. Hence, we propose a method to consolidate existing patterns of the network from two perspectives of data and model, thereby reducing the training complexity and improving the preservation of existing patterns.

4.3.1 Data-view. In order to maintain existing patterns, we can save some historical graph data, including nodes and their neighbors, and then revisit these data at each step of the incremental training. Suppose that we have a memory $\mathcal{M}$ to store some historical data. The simple random sampling strategy often has relatively large errors and unstable results. Besides, some studies [21, 32] have pointed out that the importance of different samples is different. The unimportant samples do not contribute much to the model convergence and may fail to consolidate the historical knowledge well. Therefore, we propose a step-wise sampling strategy. On the premise of ensuring the stable distribution of memory categories, select more important nodes to save, and revisit them during the process of training in the streaming scenario.

- **Hierarchical sampling on clusters:** We divide the nodes in the network into several categories, which are labels or communities of nodes. The probability of the k -th node being sampled is $p_k(v_i) = \frac{m_k}{n_k} = p$. On the one hand, it can ensure that the distribution of different types of nodes in memory and in the entire network is consistent. On the other hand, its sampling error is smaller than simple random sampling: $\sigma_{hir}(\mathcal{M}) \leq \sigma_{hir}(\mathcal{M}) + \frac{1-p}{m} \sum \frac{n_k}{n-1} (\bar{x}_k - \bar{x})^2 = \sigma_{ran}(\mathcal{M})$. In addition, it can ensure that at least one node of each type of node is stored in memory, which is more suitable for scenes with imbalanced categories in historical samples.
- **Importance-based sampling within clusters:** In hierarchical sampling, the importance of each node in a cluster is the same, which is not reasonable enough, we further propose to sample each type of nodes according to the importance of the nodes. The importance of a node v_i is designed as $\frac{1}{|N_i|} \sum_{v_j \in N_i} \mathbf{I}(y_i \neq y_j)$, where $\mathbf{I}$ is the indicator function. A node is important if its attributes are quite different from the attributes of its neighbors. These important nodes are more likely to locate at the class boundary and contribute more

to the gradient. Revisiting them can better consolidate the historical knowledge.

In the streaming scenario, we adopt the reservoir sampling algorithm [26] to extend the step-wise sampling strategy so as to update the memory $\mathcal{M}$ online, and the probability that node v_i replaces the k -th node in the memory is

$$p(v_i) = p_k(v_i) = \frac{m_k}{n_k} [1 + \alpha \cdot \frac{1}{|N_i|} \sum_{v_j \in N_i} \mathbf{I}(y_i \neq y_j)], \quad (11)$$

where α is a hyper-parameter that controls the weight of importance-based sampling strategy. Then, given the memory $\mathcal{M}$, we get the optimization goal of consolidating historical knowledge from the perspective of data replaying as

$$L_{data} = \sum_{v_i \in \mathcal{M}} l(\theta; v_i). \quad (12)$$

4.3.2 Model-view. Only saving a part of historical data, and replaying these data, because this part of the data is relatively small, is prone to the overfitting problem. With the help of saving the model, the historical model is used to constrain the current model, so as to alleviate the phenomenon of overfitting, and further improve the generalization of knowledge preservation. Using L2 regularization constraints directly on the model parameters is an overly rough method and may make the results worse since it will lead to over-smoothing or invalid constraints. So, following Elastic weight consolidation (EWC) [10], a regularization-based method in continual learning, we model the posterior distribution $p(\theta|G^{t-1})$ and derive that it is equivalent to imposing weighted regularization on parameters of the GNN model.

As $p(\theta|G^{t-1})$ is intractable, the Laplace approximation is adopted to approximate the posterior. Let $q(\theta) = p(\theta|G^{t-1})$ and assume the posterior of GNN parameters to follow a Gaussian distribution with mean given by the parameters θ^{t-1} . Since $\log q(\theta)$ is a quadratic function, we carry out Taylor Expansion on θ^{t-1} and obtain

$$\log q(\theta) = -\frac{\mathbf{F}}{2}(\theta - \theta^{t-1})^2. \quad (13)$$

So we have $p(\theta|G^{t-1}) \sim N(\theta^{t-1}, 1/\mathbf{F})$, where $\mathbf{F}$ is the Fisher Information matrix and can be computed from first-order derivatives. However, calculating $\mathbf{F}$ at each step requires traversing the entire network, and the cost of storage and calculation is high. Fortunately, we use the nodes and their neighbors in memory $\mathcal{M}$ mentioned above to estimate $\mathbf{F}$ of GNN parameters as

$$\begin{aligned} \mathbf{F} &= E_v[(\frac{\partial \log p(\theta; v)}{\partial \theta})(\frac{\partial \log p(\theta; v)}{\partial \theta})^T] \\ &= \frac{1}{n} \sum_{v \in G^{t-1}} [g(\theta; v)g(\theta; v)^T] \\ &\approx \frac{1}{m} \sum_{v \in \mathcal{M}} [g(\theta; v)g(\theta; v)^T]. \end{aligned} \quad (14)$$

By approximating the posterior, we derive the objective to preserve historical information from model view:

$$L_{model} = \lambda \sum_i \mathbf{F}_i(\theta_i - \theta_i^{t-1})^2, \quad (15)$$

which can be understood as a weighted smoothing term of GNN parameters. λ sets how important the previous information is compared to the new network. F_i indicates the importance of the i -th parameter and θ_i^{t-1} is its optimal value at the last time step. On the one hand, the regular term guarantees that the distance between the current model parameters and the historical model parameters will not deviate further. On the other hand, compared with the ordinary L2 smoothing of the parameters, the key of the objective is to add the importance of different parameters, so that the changes of GNN parameters that are important to the past network are small, which guarantees the consolidation of historical information, and unimportant parameters can be updated more drastically.

Combining the above two perspectives, we obtain the optimization goal of consolidating historical knowledge as

$$L_{existing} = L_{data} + L_{model}. \quad (16)$$

4.4 Model Optimization

Combining the above objectives, we obtain the optimization goal of our model at each time step t as

$$\begin{aligned} L &= L_{new} + L_{data} + L_{model} \\ &= \sum_{v_i \in \mathcal{M} \cup \mathcal{I}(\Delta G^t)} l(\theta; v_i) + \lambda \sum_i F_i (\theta_i - \theta_i^{t-1})^2. \end{aligned} \quad (17)$$

We use Stochastic Gradient Descent (SGD) to optimize the objective so as to obtain the parameters θ^t of the GNN at each time step t . We summarize our algorithm in Algorithm 1. Comparing with the time complexity at each time of retraining relative to $O(|V^t|)$, the complexity to minimize Equation 17 depends on the learning of the changed parts and the consolidation of existing knowledge in networks. The learning of the changing part is related to the number of affected nodes, that is $O(|\mathcal{I}(\Delta G^t)|)$. And for the consolidation of existing knowledge, its complexity $O(m)$ is irrelevant to the size of streaming networks.

Algorithm 1 Learning Algorithm of ContinualGNN at time t

Require: Network snapshot at time t : $G^t = \{V^t, E^t\}$, GNN parameterized by θ^{t-1} learned on G^{t-1}

Ensure: GNN parameterized by θ^t learned on G^t

- 1: Approximate scoring function and obtain influenced node set $\mathcal{I}(\Delta G^t)$ according to Equation 5 and Equation 9
 - 2: Load important nodes from memory $\mathcal{M}$
 - 3: Calculate parameter importance F according to Equation 14
 - 4: **for** $e = 1$ to num_epochs **do**
 - 5: Calculate loss function according to Equation 17
 - 6: Update parameters using SGD
 - 7: **end for**
 - 8: Update memory $\mathcal{M}$ using influenced node set $\mathcal{I}(\Delta G^t)$ according to Equation 11
-

5 EXPERIMENTS

In this section, we evaluate the effectiveness and efficiency of our model from multiple perspectives. Firstly, we perform node classification on four data sets to prove that our model can efficiently implement incremental learning in streaming networks. Secondly,

we conduct a case study on a synthetic data to show that our model can handle catastrophic forgetting. Finally, we conduct a detailed analysis of each part of our model.

5.1 Experimental Setup

5.1.1 Datasets. We conduct experiments on three real-world data sets and a synthetic data set.

- *Cora* [16] is a static citation network with 2708 nodes, 5429 edges, 1433 features and 7 labels. A node in the network is a paper and an edge represents a paper cited or was cited by another paper. Labels represent research fields. A synthetic dynamic network is generated, which consists of 14 time steps whose label distributions vary in different snapshots.
- *Elliptic* [29] is a bitcoin transaction network and we extract a sub-network with 31448 nodes, 34230 edges, 166 features, 2 labels and 9 time steps. A node is a transaction and an edge is viewed as a flow of bitcoins between two transactions. A node label indicates whether it is licit. The data has 9 time steps, each of whom is spaced with an interval of two weeks.
- *DBLP* [24] is a dynamic citation network and we extract a sub-network with 20000 nodes, 75706 edges, 128 features, 9 labels and 24 time steps ranging from 1993 to 2016.
- *Synthetic* consists of 3072 nodes, 14788 edges, 64 features, 2 labels and 24 time steps. Different network generation algorithms and different node attribute generation algorithms are utilized to simulate streaming graph data with different patterns over time. We use the ER graph generator [7] and the community graph generator [6], and generate nodes attributes with different normal distributions as examples. More details of simulation are introduced in Section 5.2.2.

5.1.2 Baselines. We compare our model with totally 8 baselines.

- SkipGram models: *LINE* [23] is a static network embedding model and *DNE* [5] is an extension of LINE to update node representations efficiently in dynamic environment.
- GNNs (Retrained): *GraphSAGE* [8], *GCN* [9] and *EvolveGCN* [18] are retrained in the entire network at each time step. *GraphSAGE* is also named as *RetrainedGNN*, standing as an upper bound of our model. *EvolveGCN* captures the dynamism of the graph sequence by using an RNN to evolve the GCN parameters.
- GNNs (Incremental): Simple incremental learning methods based on GraphSAGE in streaming networks. *PretrainedGNN* represents a GraphSAGE learned at the first time step and not trained any longer. *SingleGNN* is a GraphSAGE trained at every time step individually. *OnlineGNN* is a variant of GraphSAGE trained in an online manner but without any consideration for pattern detection or knowledge consolidation.
- *ContinualGNN* is our model based on GraphSAGE trained incrementally via continual learning.

5.1.3 Parameter Settings. We set the embedding size of all models in all datasets to 64. For LINE and DNE, we use both first-order and second-order proximity representations and concatenate them. Parameters of all deep models are set consistently. The number of hidden layers is 2, where the size of each layer is set to 64. For each layer, we use mean aggregator with 10 neighbors sampled.

Dataset		Cora			Elliptic			DBLP			Synthetic		
Metric		F1	Accuracy	Time	F1	Accuracy	Time	F1	Accuracy	Time	F1	Accuracy	Time
SkipGram	LINE	0.3940	0.5292	0.1510	0.5608	0.7366	5.3147	0.2234	0.3293	27.487	0.6804	0.6829	0.2715
	DNE	0.3519	0.4947	0.0750	0.5385	0.7338	0.7772	0.1979	0.3138	0.8217	0.5420	0.5454	0.0171
GNNs (Retrained)	GraphSAGE	0.6692	0.8443	0.2847	0.9273	0.9411	0.1920	0.6415	0.6699	2.6969	0.7536	0.7569	0.2756
	GCN	0.6554	0.8266	0.2803	0.9246	0.9229	0.1816	0.6425	0.6683	2.7628	0.7536	0.7575	0.2559
	EvolveGCN	0.4416	0.6014	2.7608	0.8449	0.8793	1.7532	0.5679	0.5871	6.6777	0.6297	0.6347	1.7725
GNNs (Incremental)	PretrainedGNN	0.2812	0.5362	0.0000	0.7338	0.8661	0.0000	0.1741	0.3512	0.0000	0.5517	0.5861	0.0000
	SingleGNN	0.2507	0.4186	0.0281	0.7479	0.7754	0.0267	0.3038	0.4499	0.5408	0.6445	0.6498	0.0375
	OnlineGNN	0.6018	0.7724	0.0290	0.7631	0.7911	0.0286	0.6171	0.6467	0.5304	0.6592	0.6644	0.0418
	ContinualGNN	0.6496	0.8245	0.0366	0.9035	0.9212	0.0641	0.6294	0.6685	0.4952	0.7281	0.7337	0.0884

Table 1: Averaged F1, Accuracy and Running Time per Epoch for Node Classification

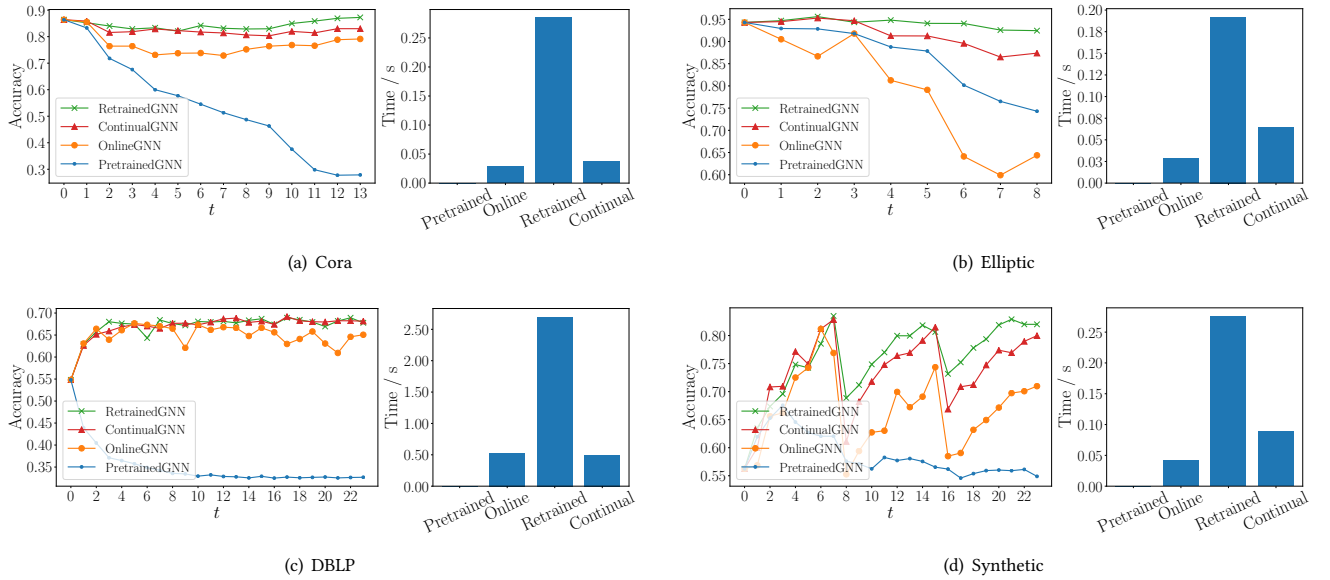


Figure 1: Node Classification on Consecutive Snapshots

For EvolveGCN, we use the version EvolveGCN-O that achieves better performance. For our model, we set $\lambda = (80, 100, 100, 200)$ and $m = (250, 500, 500, 250)$ for Cora, Elliptic, DBLP and Synthetic respectively. We use the ratio of influenced nodes of all potential nodes, including changed nodes and their L -hop neighbors, to reflect the detection threshold δ and set $\% = (0.8, 0.8, 0.8, 0.8)$.

5.2 Experimental Results

We employ four datasets for *node classification* task to demonstrate the superior of our model. Data is randomly split into training set and testing set with the ratio 7 : 3 at each time step. For supervised models, node labels are inferred directly by models. For unsupervised models, representations obtained from models are classified by Logistic Regression in sklearn package.

5.2.1 Node Classification. Table 1 shows the averaged F1 and accuracy of all baselines. Comparing with SkipGram models and

other incremental GNNs, our model, ContinualGNN achieves the best performance and gets the closest results to the upper bound, RetrainedGNN, proving that our model has great advantages in streaming networks. DNE is also an incremental learning method on streaming networks. However, it ignores node attributes when learning representations, and is not a deep model, so the result is uncompetitive compared to the GNN models. It should be noted that EvolveGCN performs poorly on Cora and Synthetic because synthetic streaming data lack temporal patterns.

Figure 1 shows the accuracy of node classification of four incremental GNNs on continuous time steps. On Cora and Elliptic, it can be seen that the results of our model can maintain a good result over time, indicating that our model can learn new knowledge while consolidating existing patterns. While the results of PretrainedGNN and OnlineGNN both declined to a certain extent, because PretrainedGNN does not learn new knowledge during network evolution, and OnlineGNN fails to maintain old knowledge

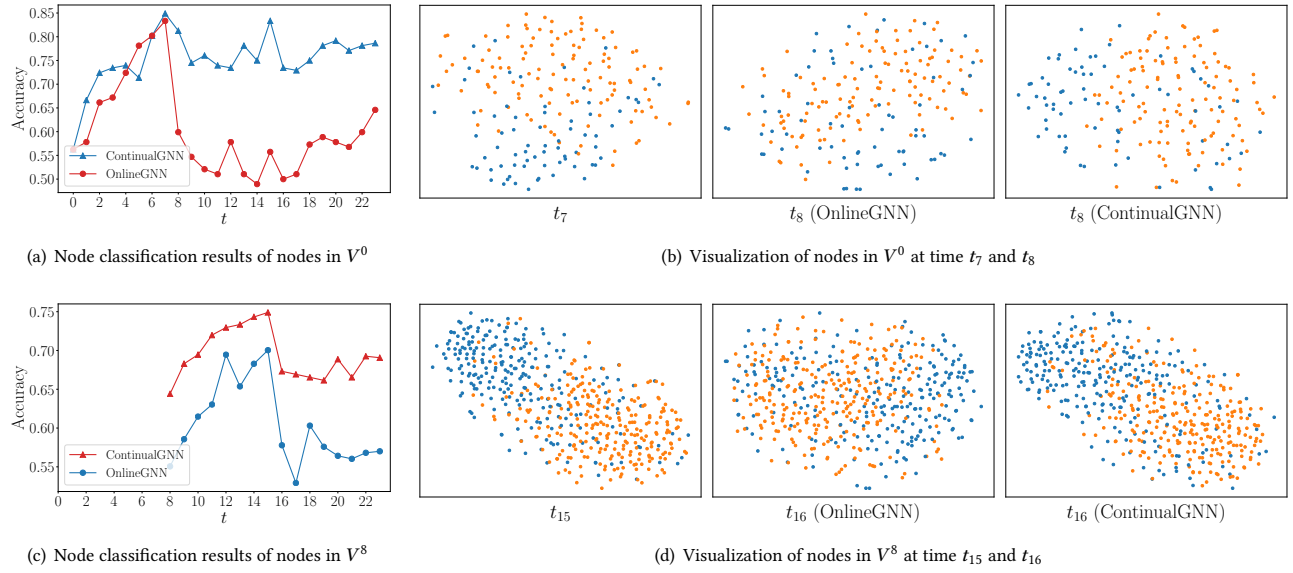


Figure 2: Case Study on Synthetic Data

well, leading to catastrophic forgetting. On DBLP, the results of PretrainedGNN will be very poor, indicating that a large number of new patterns appear after the first time step. The performance of OnlineGNN generally looks good, however, it is very unstable compared to our model, indicating that our model also has a certain smoothing ability, and can maintain stable representations. In addition, on Synthetic, due to the dramatic changes in the network, the results of all models will be attenuated to a certain extent. The overall results of OnlineGNN are getting worse but the results of our model can recover after a short-term deterioration, indicating our model integrate new and historical knowledge well.

Besides, Table 1 and Figure 1 also show running time per epoch of each model. Even though retrained GNNs like RetrainedGNN as the upper bound of our model can maintain great performance over time, their training complexity is excessively high and about 5 times to the incremental ones, which is impractical in large-scale networks. Our proposed model, ContinualGNN can be trained with high efficiency in an incremental manner so that it provides opportunities for various real-world applications.

5.2.2 Case study on Synthetic Data. We conduct experiments on the synthetic network. We construct a specific network, and then through special case study and visualization to prove that our model can deal with the problem of catastrophic forgetting in a streaming network where its distribution shifts over time.

We first introduce the generation method of the synthetic network. The ER graph generator [7] is used to generate new networks for the 0-7th time steps, the degrees of the two categories are 4 and 10, and the community graph generator [6] is used to generate new networks for the 8-23th time steps. The probability of edge generation within and between communities is 0.02 and 0.001 respectively. On the 0-15th time steps, the first dimension of the node attributes

is generated by the normal distribution $\mathcal{N}(-1, 1)$, and on the 16-23th time steps it is generated by the normal distribution $\mathcal{N}(1, 1)$. The other dimensions all follow the standard normal distribution. Through this generation method, the networks of the 0-7th and 8-15th time steps have different network topology distributions, and the networks of the 8-15th and 16-23th time steps have different node attribute distributions.

Figure 2(a) shows the performance of nodes arriving at time t_0 . As time evolves and the GNN model is updated, the classification accuracy changes. It can be seen that at the 0-7th time steps, because the distribution of the network is roughly unchanged, the accuracy rate gradually increases as the training data becomes more. But at time t_8 and t_9 , the network patterns occur changes. For OnlineGNN, training the GNN on new nodes makes the model no longer applicable to the nodes at t_0 , and catastrophic forgetting occurs, so its classification accuracy greatly decreases. On the contrary, our model, ContinualGNN, its classification accuracy can remain stable.

Figure 2(b) also shows the visualization of representations obtained at two adjacent time steps of nodes appearing at t_0 . At time t_7 , the GNN model can well separate the two types of nodes. At time t_8 , when the distribution of the network structure changes, the representations of nodes belonging to two labels through OnlineGNN cannot be distinguished, while ContinualGNN can still distinguish the two kinds of nodes. It demonstrates that when the structure distribution changes, our model can well maintain historical knowledge and avoid catastrophic forgetting.

Similarly, Figure 2(c) shows the node classification results of the nodes at t_8 with the evolution of time. Figure 2(d) shows the visualization results of the nodes of the 8-th time step at two adjacent time steps. They all prove that when the distribution of node attributes changes, ContinualGNN can consolidate the existing knowledge and maintain satisfactory results.

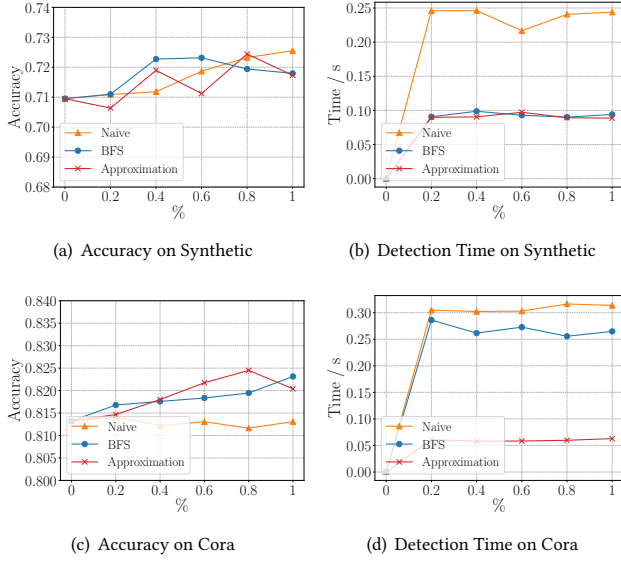


Figure 3: Analysis of New Pattern Detection

5.2.3 New Pattern Detection. We analyzed the accuracy of node classification and the running time of the detection algorithm under different detection thresholds and different detection methods during the detection of new patterns on Synthetic and Cora. *Naive* means to directly calculate the scores of all nodes in the network according to the Equation 5, *BFS* refers to calculate the scores of all neighbors of changed nodes, and *Approximation* represents our approximate calculation method.

The results are shown in Figure 3. On the one hand, it can be seen that as a larger proportion of new patterns are added during training, the accuracy will also be higher. On the other hand, it can also be seen that our *Approximation* is comparable to *Naive* and *BFS* in accuracy, indicating that our algorithm can approximate the score of the degree of node influence. But at the same time, thanks to the approximation algorithm only calculating a small number of nodes in the network, the time complexity is lower, so the effectiveness of the approximation algorithm is proved. But notice that on Synthetic, the detection time of *Approximation* and *BFS* is close and this is because the number of changed nodes and influenced nodes are similar in the network.

5.2.4 Existing Pattern Consolidation. Next, on synthetic data, we discuss the impact of different views during the consolidation of the existing patterns in streaming networks.

We first analyze from a data perspective. Figures 4(a) and Figure 4(b) respectively show the accuracy and running time of node classification on synthetic data, under different sampling strategies and different memory capacities m . On the one hand, as the memory capacity increases, the more representative nodes are saved, so the accuracy of the model is higher and the running time is longer. In order to balance the effectiveness and efficiency, we choose the memory size of 250. On the other hand, *Random sampling* refers to simple random sampling, *Hierarchical sampling* means hierarchical

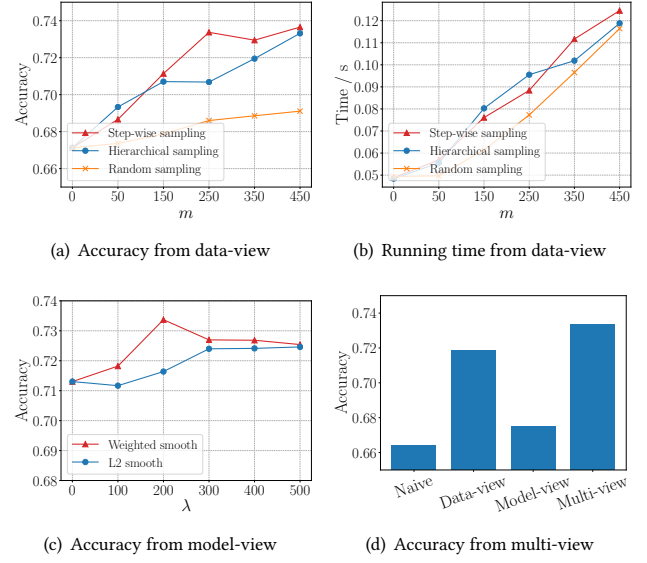


Figure 4: Analysis of Existing Pattern Consolidation

sampling without importance, and *Step-wise sampling* is our step-wise sampling strategy. It shows that the sampling strategy we designed can achieve better results, and there is not much difference in the running time of the three. This proves that the step-wise sampling strategy can select nodes that can consolidate historical knowledge more efficiently and stably in streaming networks.

We also analyze the effects of different kinds of regularization terms and different regularization weights. The results are shown in Figures 4(c). The weighted regularization method we proposed has a better effect than the L2 regularization or no constraint, that is, $\lambda = 0$. It proves that our method can better constrain the model and make it have a stronger generalization ability for knowledge preservation. At the same time, the best λ value on Synthetic is 200.

Besides, we show the effect of saving existing knowledge from different views on node classification results in Figure 4(d). It can be seen that from either data-view or model-view, the model results are improved to a certain extent. But the combination of the two can make up for each other's deficiencies and achieve the best results.

5.2.5 Scalability. Finally, we discuss the scalability of our model from two aspects. As shown in Figure 5(a), we evaluate the running time in synthetic networks with different numbers of nodes, which are collected over all training data. It shows that the running time of retraining models increases much faster than our model when the network scale becomes large. But our model can keep converging in a short time, which indicates that the scalability of our proposed model is empirically good. Then, in Figure 5(b), we observe the running time with different sizes of streaming data per time step. Obviously, the running time of our model is relative to the size of data per time step and our model have good scalability on real-world streaming networks where the streaming data per time step is often much fewer than the total scale of networks.

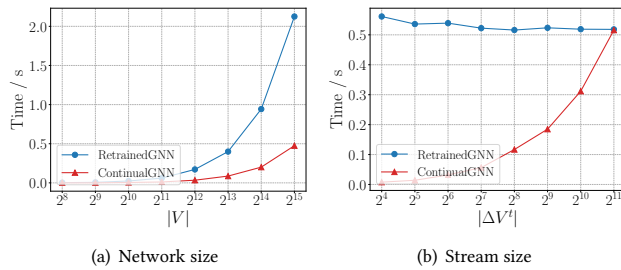


Figure 5: Scalability

6 CONCLUSION

In this paper, we address the problem of incremental learning for GNN models based on continual learning when data distribution shifts and new patterns appears over time. We designed an approximation algorithm based on traversal, which can quickly detect new patterns that may exist in a streaming network. Then, based on two complementary perspectives, we propose a method to consolidate the existing knowledge in the network, that is, constraining the current model based on a small portion of historical data and the previous model. Experiments on both real-world and synthetic networks prove that our model is more efficient than retraining models but achieves comparable results.

ACKNOWLEDGMENTS

We are thankful to Yi Li for his helpful suggestions. This work was supported by the National Natural Science Foundation of China (Grant No. 61876006).

REFERENCES

- [1] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. 2018. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 139–154.
- [2] Rahaf Aljundi, Klaas Kelchtermans, and Tinne Tuytelaars. 2019. Task-free continual learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 11254–11263.
- [3] Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. 2019. Online continual learning with no task boundaries. *arXiv preprint arXiv:1903.08671* (2019).
- [4] Songgaojun Deng, Huzefa Rangwala, and Yue Ning. 2019. Learning Dynamic Context Graphs for Predicting Social Events. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1007–1016.
- [5] Lun Du, Yun Wang, Guojie Song, Zhicong Lu, and Junshan Wang. 2018. Dynamic Network Embedding: An Extended Approach for Skip-gram based Network Embedding. In *IJCAI*. 2086–2092.
- [6] Santo Fortunato. 2010. Community detection in graphs. *Physics reports* 486, 3-5 (2010), 75–174.
- [7] Edgar N Gilbert. 1959. Random graphs. *The Annals of Mathematical Statistics* 30, 4 (1959), 1141–1144.
- [8] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*. 1025–1035.
- [9] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [10] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences* 114, 13 (2017), 3521–3526.
- [11] Jundong Li, Harsh Dani, Xia Hu, Jiliang Tang, Yi Chang, and Huan Liu. 2017. Attributed network embedding for learning in a dynamic environment. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. ACM, 387–396.
- [12] Xi Liu, Ping-Chun Hsieh, Nick Duffield, Rui Chen, Muhe Xie, and Xidao Wen. 2018. Streaming Network Embedding through Local Actions. *arXiv preprint arXiv:1811.05932* (2018).
- [13] David Lopez-Paz and Marc’Aurelio Ranzato. 2017. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*. 6467–6476.
- [14] Yao Ma, Ziyi Guo, Zhaochun Ren, Eric Zhao, Jiliang Tang, and Dawei Yin. 2018. Streaming Graph Neural Networks. *arXiv preprint arXiv:1810.10627* (2018).
- [15] Arun Mallya and Svetlana Lazebnik. 2018. Packnet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 7765–7773.
- [16] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the construction of internet portals with machine learning. *Information Retrieval* 3, 2 (2000), 127–163.
- [17] Cuong V Nguyen, Yingzhen Li, Thang D Bui, and Richard E Turner. 2017. Variational continual learning. *arXiv preprint arXiv:1710.10628* (2017).
- [18] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, and Charles E Leiserson. 2019. Evolvegcn: Evolving graph convolutional networks for dynamic graphs. *arXiv preprint arXiv:1902.10191* (2019).
- [19] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, 701–710.
- [20] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. 2017. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*. 2001–2010.
- [21] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. 2015. Prioritized experience replay. *arXiv preprint arXiv:1511.05952* (2015).
- [22] Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. 2017. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*. 2990–2999.
- [23] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. 2015. Line: Large-scale information network embedding. In *Proceedings of the 24th International Conference on World Wide Web*. International World Wide Web Conferences Steering Committee, 1067–1077.
- [24] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. 2008. Arnetminer: extraction and mining of academic social networks. In *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, 990–998.
- [25] Guillem Velicković, Petar any Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [26] Jeffrey S Vitter. 1985. Random sampling with a reservoir. *ACM Transactions on Mathematical Software (TOMS)* 11, 1 (1985), 37–57.
- [27] Junshan Wang, Yilun Jin, Guojie Song, and Xiaojun Ma. [n.d.]. EPNE: Evolutionary Pattern Preserving Network Embedding. ([n.d.]).
- [28] Geoffrey I Webb, Roy Hyde, Hong Cao, Hai Long Nguyen, and Francois Petitjean. 2016. Characterizing concept drift. *Data Mining and Knowledge Discovery* 30, 4 (2016), 964–994.
- [29] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I Weidele, Claudio Bellei, Tom Robinson, and Charles E Leiserson. 2019. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591* (2019).
- [30] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How Powerful are Graph Neural Networks? *arXiv preprint arXiv:1810.00826* (2018).
- [31] Sijie Yan, Yuanjun Xiong, and Dahua Lin. 2018. Spatial temporal graph convolutional networks for skeleton-based action recognition. In *Thirty-second AAAI conference on artificial intelligence*.
- [32] Haiyan Yin and Sinno Jialin Pan. 2017. Knowledge transfer for deep reinforcement learning with hierarchical experience replay. In *Thirty-First AAAI Conference on Artificial Intelligence*.
- [33] Friedemann Zenke, Ben Poole, and Surya Ganguli. 2017. Continual learning through synaptic intelligence. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*. JMLR. org, 3987–3995.
- [34] Lekui Zhou, Yang Yang, Xiang Ren, Fei Wu, and Yueting Zhuang. 2018. Dynamic network embedding by modeling triadic closure process. In *Thirty-Second AAAI Conference on Artificial Intelligence*.
- [35] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2847–2856.
- [36] Yuan Zuo, Guannan Liu, Hao Lin, Jia Guo, Xiaoqian Hu, and Junjie Wu. 2018. Embedding Temporal Network via Neighborhood Formation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2857–2866.